

Implement Client Script & UI Policy (Incident)

Project Documentation

1. Introduction

- **Project Title:** Implement Client Script & UI Policy (Incident)
- **Team Members:** The team and the roles / responsibilities of each member are listed below.

Name	Role	Responsibilities
Shyaam Sundhar A	Team Lead	Create the UI Policy on Incident; configure the UI Policy Action for the Urgency field.
Dhinakaran R	Member	Create Client Scripts; test list edit blocking.
Jegadheesmani B	Member	Create Client Scripts; test mandatory enforcement; test form-based update. Test successful save; test the reverse condition; prepare the conclusion

2. Project Overview

- **Purpose:** Incident records need consistent and accurate data for effective triage, routing and resolution. Relying only on user awareness and manual checks can lead to incomplete, inconsistent or incorrect data, which affects reporting accuracy, SLA compliance and service quality. This project enforces conditional field behaviour and save-time validation directly at the user-interface level in ServiceNow, using UI Policies and Client Scripts, so that Incident records meet defined business rules before they are submitted.
- **Goals:** Demonstrate how ServiceNow client-side controls can enforce data integrity on Incident records by making fields mandatory, auto-populating values, controlling field behaviour, and preventing submission when required conditions are not met.
- **Features:** Key functionalities of the solution:
 - o UI Policy "High Impact Control" that applies extra controls when Impact is 1 – High.
 - o Assignment group becomes mandatory for High impact incidents, with "Reverse if false" enabled so the change is undone automatically.
 - o UI Policy Action that makes the Urgency field read-only while Impact is High.
 - o onChange Client Script that automatically sets Urgency to High (1) when Impact is set to High and shows an information message.

- o onSubmit Client Script that blocks saving a High impact incident when Assigned To is empty and shows an error message.
- o onCellEdit Client Script that stops the State field from being edited directly in the Incident list view.

3. Architecture

The solution is built entirely on the ServiceNow Now Platform using configuration records and short JavaScript client scripts; no separate web application or custom server code is developed.

- **Frontend:** The ServiceNow platform user interface (HTML, CSS and JavaScript served by the platform) is used instead of a custom React app. The Incident form and the Incident list view are the screens users work on. UI Policies and Client Scripts run in the user's browser and control field behaviour (mandatory, read-only, auto-set values, messages) without a server round trip.
- **Backend:** The ServiceNow platform application server processes Incident inserts and updates once the client-side checks pass. No custom server-side logic (Business Rules, Script Includes) is added in this project. The configuration itself is stored as platform records: UI Policies (sys_ui_policy), UI Policy Actions (sys_ui_policy_action) and Client Scripts (sys_script_client).
- **Database:** Data is stored in the platform's relational database in the Incident table (incident); no external database such as MongoDB is used. The fields involved are:

Field (column)	Label	Role in this project
impact	Impact	Trigger field. Value 1 (High) activates the UI Policy and the scripts.
urgency	Urgency	Made read-only by the UI Policy Action; set to 1 (High) by the onChange script.
assignment_group	Assignment group	Made mandatory by the UI Policy when Impact is High.
assigned_to	Assigned To	Checked by the onSubmit script; the save is blocked if empty for High impact.
state	State	Protected from inline editing in the list view by the onCellEdit script.

4. Setup Instructions

- **Prerequisites:** Software and access required:
 - o A ServiceNow instance (for example, a Personal Developer Instance).

- o A user with administrative or configuration access (for example, the admin role).
 - o A modern web browser such as Google Chrome or Microsoft Edge.
 - o Access to the Incident application and at least one active user who can be selected in Assigned To.
- **Installation:** There is nothing to clone or install and no environment variables; the solution is created by configuring the instance in the following order:
 1. Log in to the ServiceNow instance with administrative or configuration access.
 2. Create the UI Policy: go to System UI → UI Policies and click New. Set Name = High Impact Control, Table = Incident, Active = true. In Conditions set Impact is 1 – High. Enable Reverse if false and click Submit.
 3. Add a UI Policy Action on the same policy for the Assignment group field and set Mandatory to true.
 4. Create the UI Policy Action for Urgency: in the UI Policy Actions related list of High Impact Control click New, select the Urgency field, set Read-only = true, leave Visible unchanged, and click Submit.
 5. Create the onChange Client Script: go to System UI → Client Scripts and click New. Name = Auto set urgency for high impact, Table = Incident, Type = onChange, Field name = Impact, Active = true. Paste the script from section 7 and click Submit.
 6. Create the onSubmit Client Script: Name = Prevent save if Assigned To missing, Table = Incident, Type = onSubmit, Active = true. Paste the script from section 7 and click Submit.
 7. Create the onCellEdit Client Script: Name = Prevent state change via list edit, Table = Incident, Type = onCellEdit, Field name = State, Active = true. Paste the script from section 7 and click Submit.
 8. Run the test cases in section 10 to confirm the configuration works.

5. Folder Structure

ServiceNow does not use a client/server folder tree; the solution is organised as configuration records under the System UI menu.

- **Client:** Browser-side components, all created under System UI:

Navigation path	Record name	Type	Table / Field
System UI → UI Policies	High Impact Control	UI Policy	Incident / condition: Impact is 1 – High
UI Policies → UI Policy Actions	Assignment group	UI Policy Action	Mandatory = true

Navigation path	Record name	Type	Table / Field
UI Policies → UI Policy Actions	Urgency	UI Policy Action	Read-only = true
System UI → Client Scripts	Auto set urgency for high impact	onChange	Incident / Impact
System UI → Client Scripts	Prevent save if Assigned To missing	onSubmit	Incident
System UI → Client Scripts	Prevent state change via list edit	onCellEdit	Incident / State

- **Server:** Managed by the ServiceNow platform. The Incident table (incident) and the platform processing of record insert/update are used as provided; no custom server-side files or scripts are created in this project.

6. Running the Application

There are no local servers to start; the instance is hosted in the cloud and the rules become active as soon as the records are saved with Active = true.

- Open the ServiceNow instance URL in a browser and log in.
 - **Frontend:** Go to Incident → Create New (form) or Incident → All (list). The UI Policy and Client Scripts load automatically with the form or list. No npm start command is required.
 - **Backend:** No command is required; the platform server runs as part of the hosted instance.

7. API Documentation

The project does not expose REST endpoints. Its "interfaces" are the client-side scripts and the ServiceNow client API (g_form) they call. They are documented below with their trigger, input and resulting output.

7.1 Script summary

Script name	Type	Trigger	Behaviour / example result
Auto set urgency for high impact	onChange	Impact field changes	Input: Impact = 1 (High). Output: Urgency set to 1 and the message "Urgency set to High for High impact incident." is shown.

Script name	Type	Trigger	Behaviour / example result
Prevent save if Assigned To missing	onSubmit	User saves the Incident	Input: Impact = 1 and Assigned To empty. Output: save cancelled; error shown on Assigned To: "Assigned To is mandatory for High impact incidents."
Prevent state change via list edit	onCellEdit	State edited in list	Input: any State change in the list. Output: alert "State cannot be updated using list editing. Please open the Incident." and the edit is cancelled.

7.2 onChange – Auto set urgency for high impact

```
function onChange(control, oldValue, newValue, isLoading) {
  if (isLoading || newValue == '') {
    return;
  }
  if (newValue == '1') {
    g_form.setValue('urgency', '1');
    g_form.addInfoMessage('Urgency set to High for High impact incident.');
```

7.3 onSubmit – Prevent save if Assigned To missing

```
function onSubmit() {
  if (g_form.getValue('impact') == '1' &&
    g_form.getValue('assigned_to') == '') {
    g_form.showErrorMessage(
      'assigned_to',
      'Assigned To is mandatory for High impact incidents.'
    );
    return false;
  }
  return true;
}
```

7.4 onCellEdit – Prevent state change via list edit

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
  alert(
    'State cannot be updated using list editing. Please open the Incident.'
  );
  callback(false);
}
```

7.5 Client API methods used

Method	Purpose
g_form.getValue(field)	Reads the current value of a form field (impact, assigned_to).
g_form.setValue(field, value)	Sets a field value (urgency = 1).
g_form.addInfoMessage(text)	Shows an information banner on the form.

Method	Purpose
<code>g_form.showErrorBox(field, text)</code>	Shows an error message against a specific field.
<code>return false (onSubmit) / callback(false) (onCellEdit)</code>	Cancels the save or the inline list edit.

8. Authentication

- Users sign in to the ServiceNow instance with their platform credentials (or the single sign-on method configured for the instance); session handling is done by the platform. This project does not add any custom tokens or session logic.
- Authorization is role based. Only users with administrative or configuration access can create or change UI Policies, UI Policy Actions and Client Scripts. Regular Incident users only experience the resulting form and list behaviour.
- Standard platform access controls (ACLs) continue to decide who can read and write Incident records. UI Policies and Client Scripts improve data quality in the interface but are not a security boundary.

9. User Interface

The following UI behaviour is delivered on the Incident form and list (described in text; no screenshots are embedded in this document):

- **Impact set to High:** the UI Policy "High Impact Control" activates; Assignment group becomes mandatory and Urgency becomes read-only.
- **Urgency auto-update:** Urgency changes to High (1) automatically and an information message "Urgency set to High for High impact incident." appears at the top of the form.
- **Save validation:** if Assigned To is empty when saving a High impact incident, the record is not saved and an error appears beside the Assigned To field.
- **Impact changed from High to another value:** Reverse if false removes the mandatory and read-only behaviour, so the fields return to normal.
- **List view:** double-clicking the State cell of an Incident in the list shows an alert and leaves the value unchanged; State can still be changed from the Incident form.

10. Testing

- **Strategy:** Manual functional testing directly in the ServiceNow instance. Each client-side rule is exercised through the Incident form and the Incident list, including the positive case, the blocked case and the reverse condition.

- **Tools:** ServiceNow Incident form (Incident → Create New / Open) and Incident list view (Incident → All) in a web browser, using an account with the required access.

#	Test	Steps and expected result	Tester
1	Mandatory Enforcement	Create a new Incident, set Impact = High, leave Assigned To empty and submit. Expected: the Incident is not saved and an error message is displayed; UI Policy actions and urgency update work.	Dhinakaran R
2	Successful Save	Fill Assigned To and submit. Expected: the Incident saves without errors; urgency auto-set, read-only fields and state restriction still behave as configured.	Jegadheesmani B
3	Reverse Condition	Open an Incident with Impact = High and change Impact to Medium. Expected: mandatory behaviour is removed, Urgency is editable again, and the record saves without errors.	Jegadheesmani B
4	List Edit Blocking	In Incident → All, double-click the State field. Expected: an alert says State cannot be updated via the list and the value stays unchanged.	Shyaam Sundhar A
5	Form-Based Update	Open the Incident form, change State and click Update. Expected: the change is saved successfully; other rules still apply.	Dhinakaran R

11. Screenshots or Demo

This document does not embed screenshots. A text walkthrough of the demo flow is given below; a demo link or screenshots can be attached separately by the team if required.

1. Open Incident → Create New and set Impact to High. Urgency changes to High and becomes read-only, the information message appears, and Assignment group is marked mandatory.
2. Leave Assigned To empty and click Submit. The save is blocked with an error on Assigned To.
3. Select a user in Assigned To and Submit. The Incident is saved.
4. Open the Incident again and change Impact to Medium. The mandatory and read-only behaviour is reversed and the record can be saved.

5. Go to Incident → All and try to edit State by double-clicking the cell. An alert is shown and State is unchanged; then change State on the form and click Update to confirm it saves.

12. Known Issues

- Client-side only: UI Policies and Client Scripts run in the browser interface. Records created or updated through imports, web service/API calls or server-side automation do not trigger them, so the same rules are not enforced there.
- The onCellEdit script blocks every inline edit of State for all users; there is no role-based exception.
- The onChange script only sets Urgency when Impact is High (value 1); Urgency is not reset when Impact changes to another value, it is only unlocked.
- The mandatory rule is split across two fields: the UI Policy makes Assignment group mandatory, while the onSubmit script checks Assigned To. Both apply when Impact is High, and test documentation should refer to both.
- The scripts use hard-coded choice values ('1') and fixed message text, so changes to the Impact choice list or wording require editing the scripts.

13. Future Enhancements

- Add a server-side Data Policy or Business Rule so the same validation also applies to integrations, imports and API updates.
- Allow role-based exceptions to the State list-edit restriction (for example for incident managers).
- Reset or recalculate Urgency when Impact changes to Medium or Low, and extend the rules to Priority and SLA-related fields.
- Move message text to UI Messages / system properties for easier maintenance and translation.
- Apply the same pattern to other tables such as Problem and Change, and package the configuration in an Update Set or application for deployment across instances.
- Automate the test cases with the ServiceNow Automated Test Framework (ATF).